



System Design Basics

Microservices Design with Java

Architecture Thinking for Scalable Systems



Session Structure

1. What is System Design?
2. Scalability & Performance Basics
3. Monolith vs Microservices
4. Microservices Architecture
5. Communication Patterns
6. Data Management
7. Java Ecosystem & Spring Boot
8. Real-world architecture case study

What is System Design?

System Design means:

Designing scalable, reliable, maintainable systems.

Focus areas:

- Performance
- Scalability
- Availability
- Consistency
- Maintainability

Functional vs Non-Functional Requirements

Functional:

- User login
- Payment processing

Non-Functional:

- Handle 1M users
- 99.99% availability
- <200ms latency



Scalability Types

Vertical Scaling:

Increase server power.

Horizontal Scaling:

Add more servers.

Modern systems:

Prefer horizontal scaling.



Key Design Concepts

- Load balancing
- Caching
- Database indexing
- Sharding
- Replication
- Asynchronous processing



Monolith Architecture

Single deployable unit.

Pros:

- Simple deployment
- Easier debugging

Cons:

- Hard to scale
- Tight coupling
- Slow deployments



Microservices Architecture

Multiple independent services.

Each service:

- Owns its data
- Deployable independently
- Loosely coupled



Microservices Benefits

- Independent scaling
- Faster development
- Fault isolation
- Technology flexibility



Microservices Challenges

- Network latency
- Distributed transactions
- Monitoring complexity
- Data consistency



Basic Microservices Architecture

Client → API Gateway → Services → Databases

Each service:

Own DB.



API Gateway

Responsibilities:

- Authentication
- Routing
- Rate limiting
- Aggregation



Service Communication

1. Synchronous (REST, HTTP)
2. Asynchronous (Kafka, RabbitMQ)

REST Communication (Spring Boot)

```
@RestController
@RequestMapping("/users")
class UserController {

    @GetMapping("/{id}")
    public User getUser(@PathVariable Long id) {
        return new User(id, "John");
    }
}
```



Asynchronous Messaging Example

Using Kafka (Conceptual):

Service A → Publish Event

Service B → Consume Event

Benefits:

Loose coupling.



Database Per Service

Each microservice:

Has its own database.

Avoid:

Shared database between services.



CAP Theorem

Consistency

Availability

Partition tolerance

In distributed systems:

Choose 2 out of 3.

THIRAVUGAL



Caching Layer

Use:

- Redis
- In-memory caching

Reduces DB load.

THIRAVUGAL



Load Balancer

Distributes traffic across instances.

Examples:

- Nginx
- AWS ELB



Circuit Breaker Pattern

Prevents cascading failures.

Example:

If Service B fails → Stop calling temporarily.

Used in:

Resilience4j, Hystrix.



Service Discovery

Dynamic service location.

Tools:

- Eureka
- Consul



Distributed Logging

Centralized logging:

- ELK Stack
- Splunk

Helps debugging distributed systems.



Docker & Containerization

Each service runs in container.

Benefits:

- Isolation
- Portability
- Easy deployment



Kubernetes Basics

- Pod
- Deployment
- Service
- Auto-scaling

THIRAVUGAL



Microservices in Java Ecosystem

Common stack:

- Spring Boot
- Spring Cloud
- Hibernate/JPA
- REST APIs
- Kafka



Example Microservice Structure

User Service:

- Controller
- Service
- Repository
- Entity



Example Layered Architecture

Controller → Service → Repository → DB

Separation of concerns.

Real World Case Study — E-Commerce

Services:

- User Service
- Product Service
- Order Service
- Payment Service
- Inventory Service

Each service independent.



Order Flow Example

1. User places order
2. Order service validates
3. Inventory reserved
4. Payment processed
5. Event published



Scalability Design Example

High traffic:

- Add more service instances
- Use load balancer
- Cache frequent queries



Data Consistency

Two models:

- Strong consistency
- Eventual consistency

Microservices usually:
Eventual consistency.



Saga Pattern

Manage distributed transactions.

Two types:

- Choreography
- Orchestration



Security in Microservices

- JWT authentication
- OAuth2
- API Gateway security



Performance Optimization

- Connection pooling
- Caching
- Database indexing
- Async processing



Common System Design Mistakes

- Overengineering
- Ignoring failure scenarios
- Tight coupling
- No monitoring
- No scalability planning



Interview Strategy

Clarify:

1. Traffic volume
2. Read vs write ratio
3. Consistency needs
4. Data size
5. Failure handling

Then:

Design step by step.



System Design Thinking Template

1. Requirements
2. High-level design
3. Component breakdown
4. Database design
5. Scaling strategy
6. Failure handling
7. Trade-offs



Summary

System Design builds:

- Architectural thinking
- Scalability awareness
- Distributed system understanding
- Microservices knowledge

Master:

- API Gateway
- Load balancing
- Database design
- Event-driven systems
- Spring Boot microservices